به نام خدا

تهیه کننده: ابراهیم صدیقی شماره دانشجویی: ۹۹۳۱۰۹۸

پاسخ به سوالات مرحله اول آزمایش

1- بیشترین سهم در زمان اجرا را حلقه اصلی (با تعداد پیمایش VERYBIG) دارد چرا که انجام عملیات های این حلقه آن هم به تعداد زیاد زمان گیر خواهد بود.
2- به دفعات تکرار کردن آزمایش، باعث می شود تا زمان های هر بار اجرای حلقه را بدانیم تا در صورتی که یکبار اجرای حلقه به دلایل مختلف زمان اجرایش با بقیه فرق کرد، آن را داده دور افتاده به حساب بیاوریم و در نتیجه نهایی که از میانگین باقی زمان ها بدست می آید لحاظ نکنیم.
3- حالت دیباگ برای زمانی استفاده می شود که در حال توسعه برنامه هستیم اما حالت release هنگامی استفاده می شود که برنامه را در حالت نهایی استفاده می کنیم. به همین دلیل، در زمان دیباگ، کامپایلر تعدادی debug symbol برای متغیر ها درست می کند اما در حالت release یک سری بهینه سازی بر روی کد دانجام می شود. برای مثال در حالت دیباگ از کدهای اضافه ای استفاده می شود که قابلیت break point گذاشتن برای کاربر را فراهم می کند اما در حالت release با بهینه سازی که انجام می شود، این کد ها به همراه دیگر متغیرها حذف می شود و به همین دلیل در حالت release سرعت بیشتری نسبت به حالت دیباگ خواهیم داشت.
4- چون هر پیمایش این حلقه مستقل از دیگری انجام می شود، می توانیم از روش تجزیه task parallelism استفاده کنیم تا تعداد پیمایش های کوچکتر توسط نخ های بیشتر انجام شود و در نهایت نتیجه نهایی آنها با هم جمع شود که این خود استفاده از الگوی موازی سازی master slave است. به این صورت که ابتدا توسط نخ ارباب به هر نخ برده تعداد مشخصی از پیمایش اصلی تکلیف می شود و در نهایت بعد از انجام اینها، نخ ارباب نتایج برده ها را با یکدیگر جمع می کند.

تحلیل اجرای برنامه با اجرای دستورات مختلف در مرحله دوم

- ابتدا برنامه اصلی را بدون موازی سازی اجرا می کنیم که نتایج زیر بدست می آید:

```
D:\Programming\C++\Projects\MCP_LAB_1\x64\Release\MCP_LAB_1.exe
Serial Timings for 50000 iterations

Time Elapsed: 2.739300 Secs, Total = 30.656747, Check Sum = 50000
Time Elapsed: 2.758504 Secs, Total = 30.656747, Check Sum = 50000
Time Elapsed: 2.755194 Secs, Total = 30.656747, Check Sum = 50000
Time Elapsed: 2.759942 Secs, Total = 30.656747, Check Sum = 50000
Time Elapsed: 2.746355 Secs, Total = 30.656747, Check Sum = 50000
Time Elapsed: 2.731592 Secs, Total = 30.656747, Check Sum = 50000
Time Elapsed: 2.752724 Secs, Total = 30.656747, Check Sum = 50000
Time Elapsed: 2.724239 Secs, Total = 30.656747, Check Sum = 50000
Time Elapsed: 2.757462 Secs, Total = 30.656747, Check Sum = 50000
Time Elapsed: 2.738170 Secs, Total = 30.656747, Check Sum = 50000
```

زمان اجرای برنامه در حالت سریال تقریبا برابر با 2.74 ثانیه است.

- بعد از فعالسازی open MP و اضافه کردن راهنمای for ، نتایج زیر برای بند 4 بدست می آید:

```
Serial Timings for 50000 iterations

Time Elapsed: 19.626144 Secs, Total = 59.891398, Check Sum = 49717
Time Elapsed: 22.369808 Secs, Total = 63.225245, Check Sum = 49729
Time Elapsed: 22.989447 Secs, Total = 63.830773, Check Sum = 49647
Time Elapsed: 23.254456 Secs, Total = 47.558803, Check Sum = 49721
Time Elapsed: 24.197604 Secs, Total = 62.547767, Check Sum = 49749
Time Elapsed: 23.317206 Secs, Total = 56.995951, Check Sum = 49696
Time Elapsed: 23.694757 Secs, Total = 66.466856, Check Sum = 49550
Time Elapsed: 23.981237 Secs, Total = 61.238795, Check Sum = 49598
Time Elapsed: 23.394811 Secs, Total = 52.610853, Check Sum = 49702
Time Elapsed: 24.493269 Secs, Total = 80.248381, Check Sum = 49594


D:\Programming\C++\Projects\MCP_LAB_1\x64\Release\MCP_LAB_1.exe (process 10928) exited with code 0.
Press any key to close this window . . .
```

در این مرحله، می بینیم که زمان زیادی طول می کشد تا برنامه اجرا شود. همچنین به دلیل وجود شرایط مسابقه، مقادیر متفاوتی برای total و check sum به وجود می آید. کد این قسمت با نام parallel.cpp به پیوست ارسال شده است.

- بعد از اضافه کردن private به راهنمای for می بینیم که همچنان مشکل شرایط مسابقه وجود دارد. زیرا همچنان دسترسی همزمان به متغیر های total و sum وجود دارد. اما زمان اجرای برنامه اکنون بسیار کمتر شده است. نتیجه اجرای بند 6 دستورکار در صفحه بعد آورده شده:

```
D:\Programming\C++\Projects\MCP_LAB_1\x64\Release\MCP_LAB_1.exe
Serial Timings for 50000 iterations

Time Elapsed: 0.645154 Secs, Total = 30.585744, Check Sum = 49888
Time Elapsed: 0.638774 Secs, Total = 30.588812, Check Sum = 49903
Time Elapsed: 0.637208 Secs, Total = 30.591390, Check Sum = 49910
Time Elapsed: 0.638105 Secs, Total = 30.605646, Check Sum = 49904
Time Elapsed: 0.638998 Secs, Total = 30.602003, Check Sum = 49915
Time Elapsed: 0.638043 Secs, Total = 30.606849, Check Sum = 49893
Time Elapsed: 0.638723 Secs, Total = 30.572013, Check Sum = 49912
Time Elapsed: 0.635548 Secs, Total = 30.591585, Check Sum = 49896
Time Elapsed: 0.636994 Secs, Total = 30.588802, Check Sum = 49917
Time Elapsed: 0.637341 Secs, Total = 30.614418, Check Sum = 49923
```

کد مربوط به این قسمت با نام parallel_private.cpp به پیوست ارسال شده است.

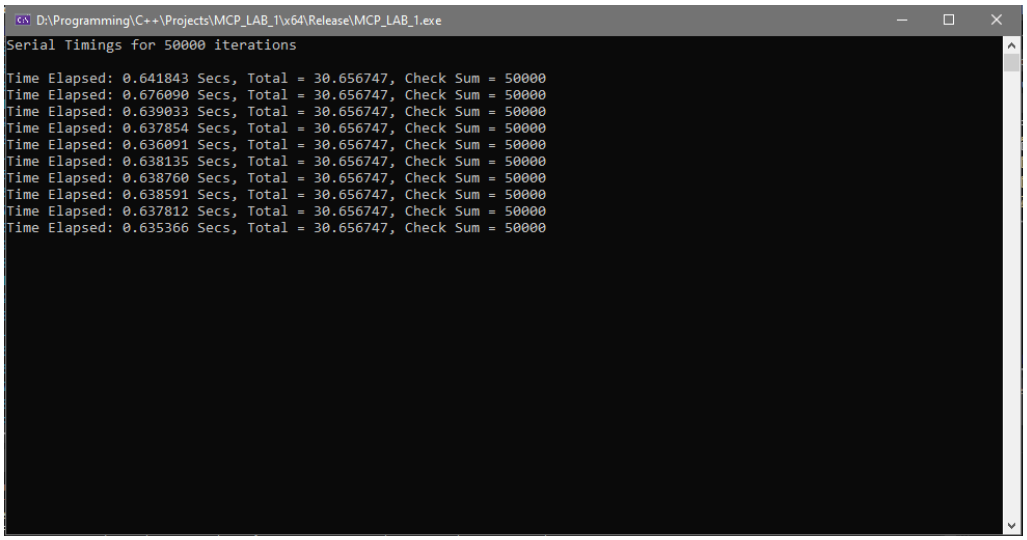
- در بند 7، راهنمای critical را بالای متغیرهای sum و total اضافه می کنیم. نتایج زیر حاصل می شود:

```
D:\Programming\C++\Projects\MCP_LAB_1\x64\Release\MCP_LAB_1.exe
Serial Timings for 50000 iterations

Time Elapsed: 0.663581 Secs, Total = 30.656747, Check Sum = 50000
Time Elapsed: 0.637007 Secs, Total = 30.656747, Check Sum = 50000
Time Elapsed: 0.639038 Secs, Total = 30.656747, Check Sum = 50000
Time Elapsed: 0.636598 Secs, Total = 30.656747, Check Sum = 50000
Time Elapsed: 0.635446 Secs, Total = 30.656747, Check Sum = 50000
Time Elapsed: 0.638561 Secs, Total = 30.656747, Check Sum = 50000
Time Elapsed: 0.638234 Secs, Total = 30.656747, Check Sum = 50000
Time Elapsed: 0.636725 Secs, Total = 30.656747, Check Sum = 50000
Time Elapsed: 0.635709 Secs, Total = 30.656747, Check Sum = 50000
Time Elapsed: 0.639031 Secs, Total = 30.656747, Check Sum = 50000
```

می بینیم که در این قسمت، همگام سازی بین نخ ها به درستی انجام شده و هر دو متغیر total و sum مقادیر درستی را اتخاذ کرده اند. زمان اجرای برنامه نیز تا حد زیادی نسبت به حالت سریال کاهش یافته و در حدود 0.63 ثانیه است. کد مربوط به این قسمت با نام parallel_critical.cpp به پیوست ارسال شده است.

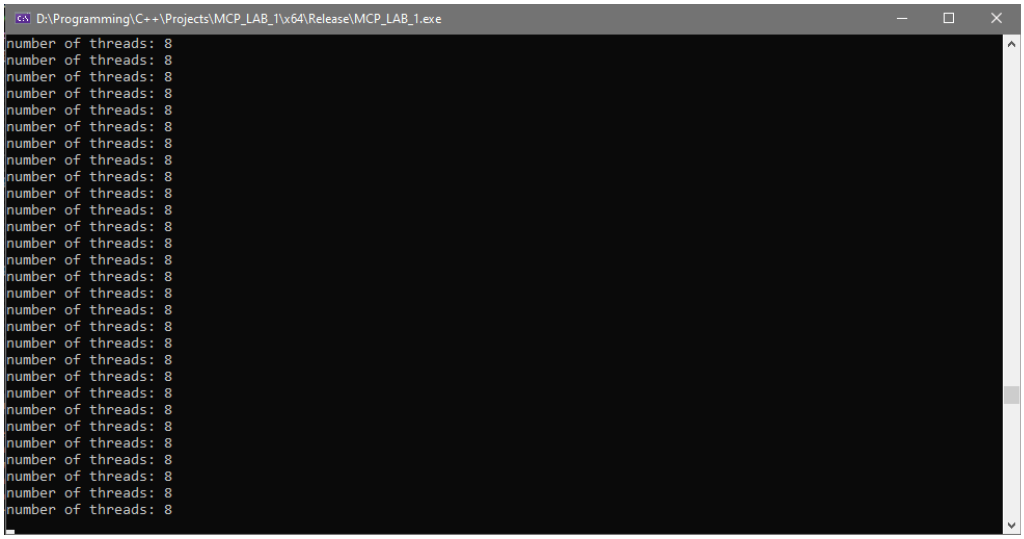
- با اضافه کردن reduction به راهنمای for، نتایج زیر برای بند 9 بدست می آید:

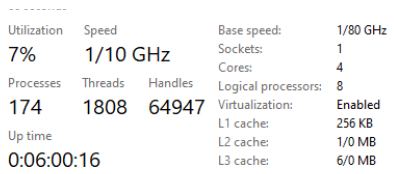


می بینیم که در این حالت نیز همگام سازی به درستی بین نخ ها اجرا شده و زمان اجرای برنامه نیز تقریبا 0.63 ثانیه است که با زمان بدست آمده در قسمت قبلی تفاوت زیادی ندارد. کد این قسمت با نام parallel_reduction.cpp به پیوست ارسال شده است.

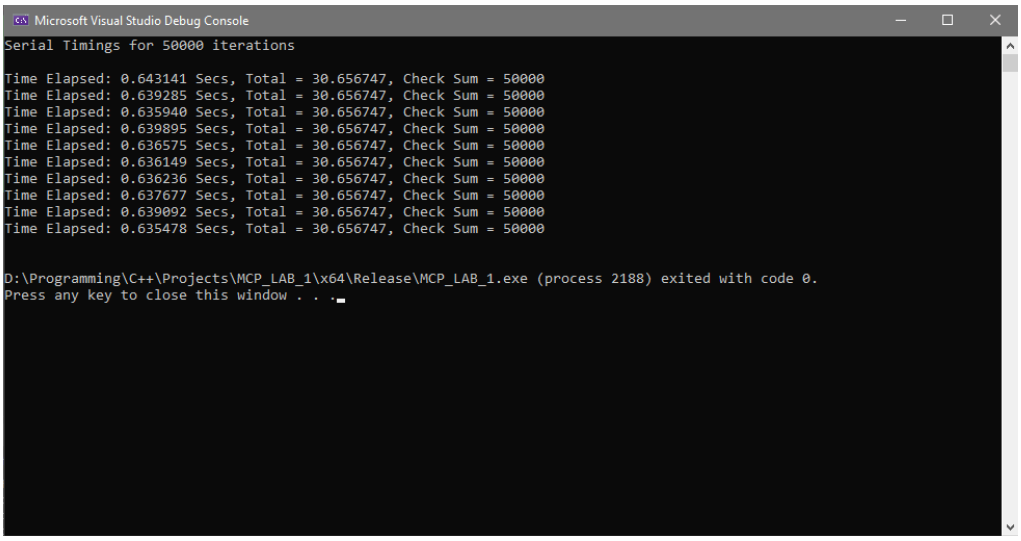
پاسخ سوالات مرحله دوم

1- وقتی با دستور omp_get_num_threads تعداد نخ های استفاده شده در ناحیه موازی را چاپ می کنیم، عدد 8 چاپ می شود. با بررسی اطلاعات سیستم خودم نیز می بینیم که تعداد هسته های پردازنده برابر با 4 و تعداد پردازنده های منطقی برابر 8 است:





2- بله. اما باید توجه کرد که دستور atomic برای عملگر += کار می کند. به همین خاطر در کدی که در پیوست با نام parallel_atomic.cpp ارسال شده است، این نکته رعایت شده. نتایج اجرای برنامه نیز به صورت زیر است:



می بینیم که برنامه در این حالت به درستی اجرا می شود و زمان اجرای آن نیز تقریبا همان 0.63 ثانیه است.

3- همانطور که اشاره شده، زمان های اجراهای ذکر شده برای این تعداد پیمایش تفاوتی با یکدیگر ندارند، اما هر چه تعداد پیمایش بیشتر شود، استفاده از reduction بهتر و سریع تر خواهد بود، زیرا با افزایش تعداد پیمایش روش ناحیه بحرانی دارای سربار زمانی قفل شدن نخ ها خواهد شد اما در روش reduction این سربار وجود نخواهد داشت و تنها در انتها مقادیر با هم جمع خواهند شد که باعث سریع تر شدن زمان اجرای برنامه می شود.

لازم به ذکر است که تسریع بدست آمده برابر است با:

$$speedup = \frac{T_s}{T_p} = \frac{2.74}{0.63} = 4.35$$